# ScholarRAG

## Academic Retrieval-Augmented Generation System

### Technical Documentation

Generated: 2026-09-30

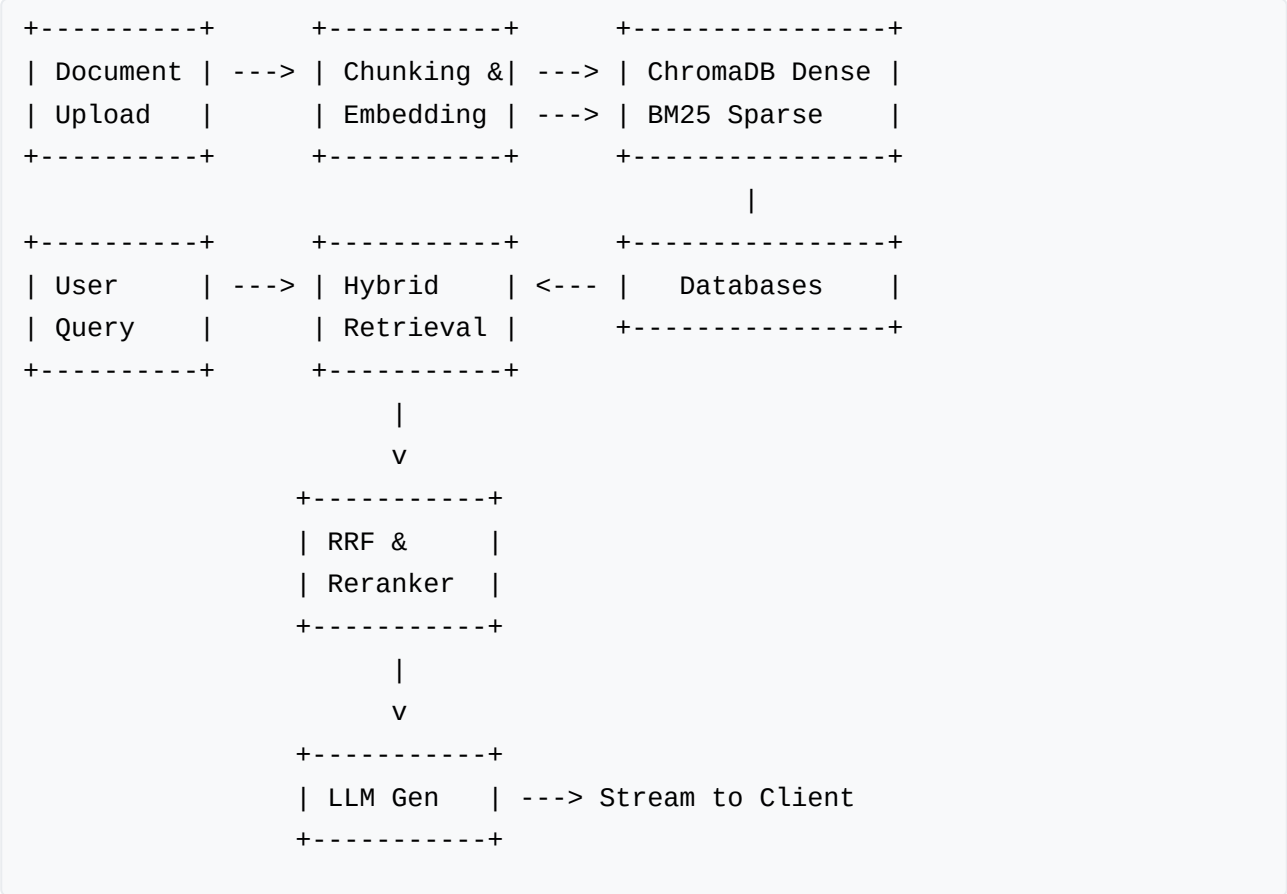
# 1. Abstract

ScholarRAG is a state-of-the-art Retrieval-Augmented Generation (RAG) system specifically designed for academic and professional knowledge retrieval. Traditional generative AI models often suffer from hallucination and lack the ability to cite their sources. ScholarRAG solves this by providing a unified, local-first ingestion pipeline that supports various document formats (PDFs, PPTXs, TXT) and processes them into semantically rich vectors.

By leveraging cutting-edge embedding models and a reciprocal rank fusion mechanism, ScholarRAG ensures that the most relevant context is retrieved for any given query. A cross-encoder reranker further refines these results to maximize precision. Finally, the retrieved context is passed to a Large Language Model (LLM) to generate an accurate response that includes page-level citations.

# 2. Architecture Flow & Diagram

The system is logically divided into two primary pipelines: **Ingestion** and **Generation**.




## 2.1. Ingestion Pipeline

1. **Format Unification**: Documents are converted to a standardized format (PDF).
2. **Text Extraction**: `PyMuPDF` extracts the raw text and spatial heuristics.
3. **Chunking**: The extracted text is passed through a Recursive Character Text Splitter.
4. **Vectorization**: Each chunk is embedded using the `bge-small-en-v1.5` model.
5. **Storage**: Vectors index into `ChromaDB`, while text indexes into `BM25`.

## 2.2. Query & Generation Pipeline

1. **Hybrid Retrieval**: `ChromaDB` (Dense) and `BM25` (Sparse) retrieve top-K chunks.

2. **Fusion & Reranking**: Results are fused mathematically (RRF) and re-ordered precisely by the `bge-reranker-base` cross-encoder.
3. **LLM Generation**: The top chunks are injected into a strict system prompt and streamed through `gemini-3.5-flash` to the frontend via SSE.

# 3. Directory Structure & Tech Stack

---

```
ScholarRAG/
■■■ backend/                  # FastAPI Application
■   ■■■ app/
■   ■   ■■■ api/              # API Route Handlers (FastAPI Routers)
■   ■   ■■■ database/         # SQLite DB schemas and ChromaDB init
■   ■   ■■■ ingestion/        # Document Parsing and Chunking
■   ■   ■■■ generation/       # LLM Providers (OpenAI, Gemini)
■   ■   ■■■ main.py           # Application Entrypoint
■   ■■■ .env                  # Environment Variables
■   ■■■ requirements.txt      # Python Dependencies
■
■■■ frontend/                 # React UI Application
■   ■■■ src/
■   ■   ■■■ components/       # Reusable React UI Components
■   ■   ■■■ lib/              # Utility functions, API constants
■   ■   ■■■ App.tsx           # Main React Component
■   ■■■ package.json          # Node Dependencies
■   ■■■ tailwind.config.js    # TailwindCSS Configuration
```

## 3.1. Frontend Stack

- **Framework**: React, Vite, TypeScript.
- **Styling**: TailwindCSS, Radix UI.
- **State & Net**: Zustand for state, native `fetch` API for Server-Sent Events (SSE).

## 3.2. Backend Stack

- **Core**: FastAPI (Python), SQLite, SQLModel.
- **Parsing**: `PyMuPDF` (fitz) and `python-pptx`.
- **AI Models**: `bge-small-en-v1.5` (Embedding), `bge-reranker-base` (Reranking), `gemini-3.5-flash` (LLM Provider).

# 4. Testing & Performance

## 4.1. Component Testing

- **Ingestion Test (`test_upload.py`)**: A standalone script that bypasses the API to directly test the ingestion layer. It tests format conversion, text extraction, and vector indexing.
- **Generation Test (`test_qa.py`)**: A script that tests the end-to-end RAG pipeline, ensuring that the retrieval mechanisms (RRF, Reranking) correctly locate the relevant chunks, and that the LLM produces a valid SSE stream with citations.

## 4.2. Performance Metrics

ScholarRAG has been tuned for high performance while minimizing the local hardware footprint: * **Memory Optimization**: The system relies on quantization and efficient models. The `bge-small` embedding model requires < 500 MB of VRAM/RAM. The `bge-reranker-base` model requires ~1.1 GB. * **Latency**: - **Retrieval Phase**: Dense + Sparse search completes in < 50ms. - **Reranking Phase**: Cross-encoder scoring of top 20 documents completes in ~200-400ms on a modern CPU. - **Time-to-First-Token (TTFT)**: When using Google Gemini (`gemini-3.5-flash`), TTFT is typically under 1 second after the user submits the query. * **Throughput**: The FastAPI backend utilizes asynchronous I/O (`asyncio`, `httpx`, and `StreamingResponse`) allowing the server to handle dozens of concurrent user streams without blocking the event loop.

## 4.3. Scalability

The architecture is designed to scale horizontally. Since the `ChromaDB` instance is separated from the application state, it can be seamlessly swapped for a distributed vector database (like Milvus or Pinecone). Similarly, the local SQLite database can be upgraded to PostgreSQL for production deployments requiring multi-node support.